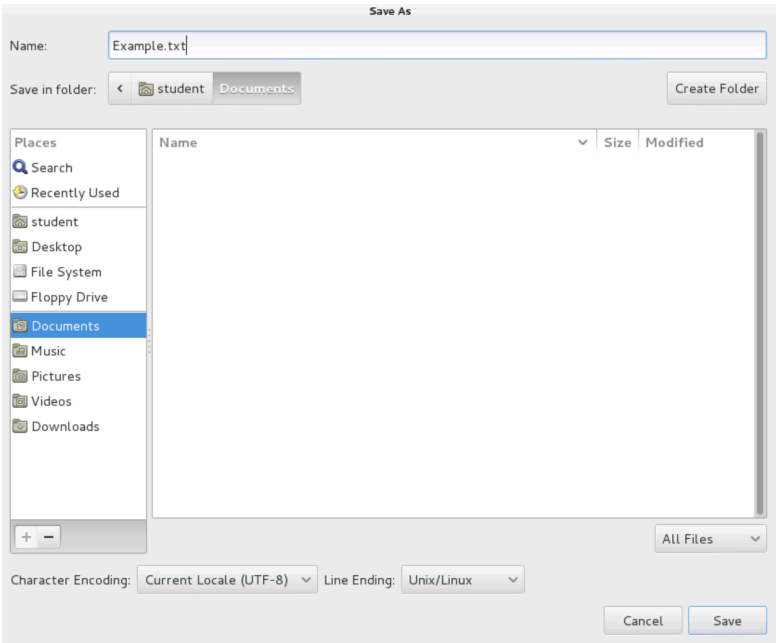
Applying Encryption and Hashing Algorithms for Secure Communications

This lab walks through different hashing algorithms such as MD5, SHA1, and SHA256 and how to use them to encrypt text files. It also introduces GPG keys and how to share keys to successfully encrypt and then decrypt a text file using the key.

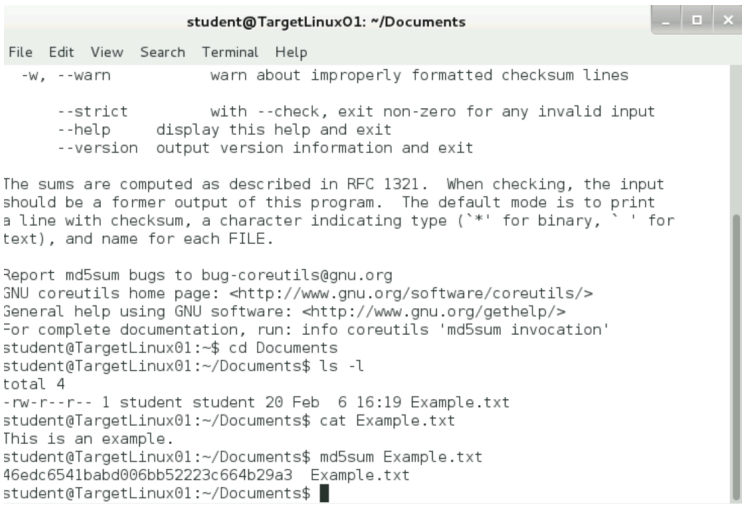
SECTION 1 - HANDS-ON DEMONSTRATION

Part 1: Create a Text File on Linux

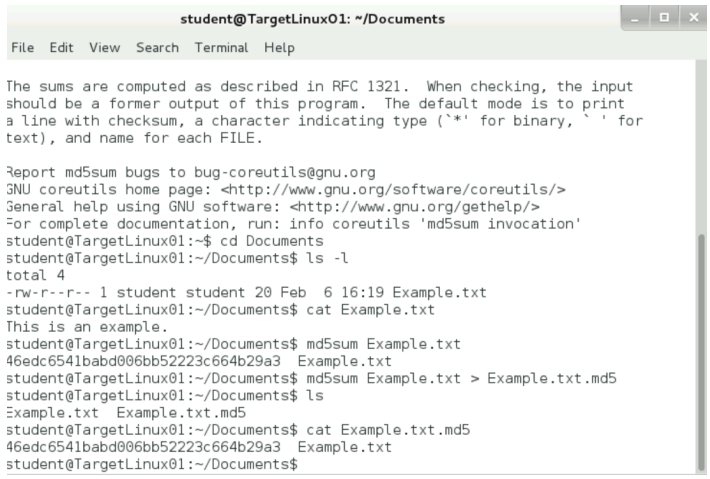


Screenshot 1: Example text file created and saved to the Documents folder on TargetLinux01.

Part 2: Create a MD5sum and a SHA1sum Hash String

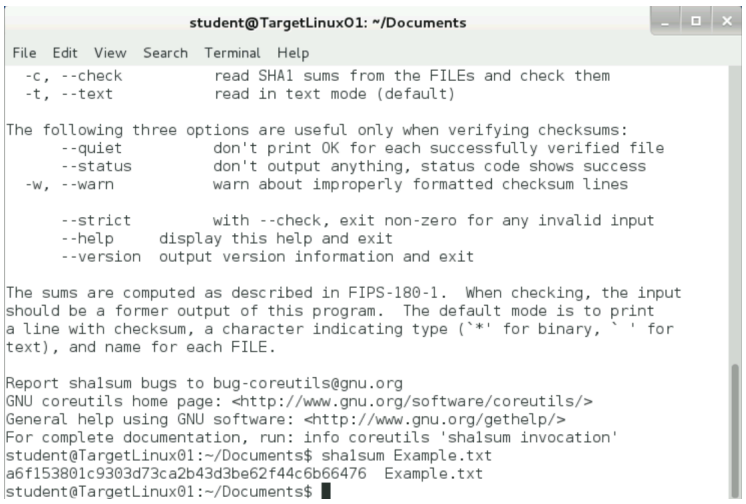


Screenshot 2: MD5 hash value for `Example.txt` file.



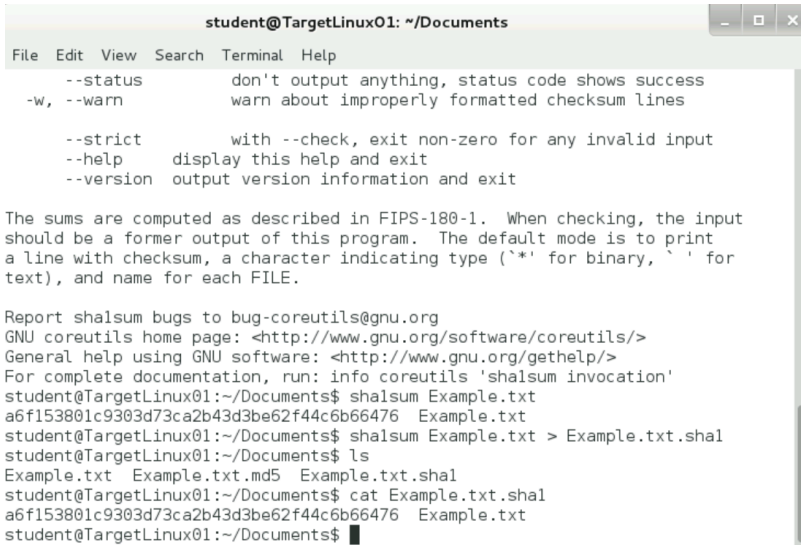
```
student@TargetLinux01: ~/Documents
File  Edit  View  Search  Terminal  Help
The sums are computed as described in RFC 1321.  When checking, the input
should be a former output of this program.  The default mode is to print
a line with checksum, a character indicating type (`*' for binary, ` ' for
text), and name for each FILE.

Report md5sum bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'md5sum invocation'
student@TargetLinux01:~$ cd Documents
student@TargetLinux01:~/Documents$ ls -l
total 4
-rw-r--r-- 1 student student 20 Feb  6 16:19 Example.txt
student@TargetLinux01:~/Documents$ cat Example.txt
This is an example.
student@TargetLinux01:~/Documents$ md5sum Example.txt
46edc6541babd006bb52223c664b29a3  Example.txt
student@TargetLinux01:~/Documents$ md5sum Example.txt > Example.txt.md5
student@TargetLinux01:~/Documents$ ls
Example.txt  Example.txt.md5
student@TargetLinux01:~/Documents$ cat Example.txt.md5
46edc6541babd006bb52223c664b29a3  Example.txt
student@TargetLinux01:~/Documents$
```

Screenshot 3: Verification that `Example.txt.md5` was created, saved to Documents, and holds the MD5 hash for `Example.txt`.



```
student@TargetLinux01: ~/Documents
File  Edit  View  Search  Terminal  Help
  -c, --check          read SHA1 sums from the FILEs and check them
  -t, --text           read in text mode (default)

The following three options are useful only when verifying checksums:
      --quiet          don't print OK for each successfully verified file
      --status         don't output anything, status code shows success
  -w, --warn           warn about improperly formatted checksum lines

      --strict         with --check, exit non-zero for any invalid input
      --help     display this help and exit
      --version  output version information and exit

The sums are computed as described in FIPS-180-1.  When checking, the input
should be a former output of this program.  The default mode is to print
a line with checksum, a character indicating type (`*' for binary, ` ' for
text), and name for each FILE.

Report sha1sum bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'sha1sum invocation'
student@TargetLinux01:~/Documents$ sha1sum Example.txt
a6f153801c9303d73ca2b43d3be62f44c6b66476  Example.txt
student@TargetLinux01:~/Documents$
```

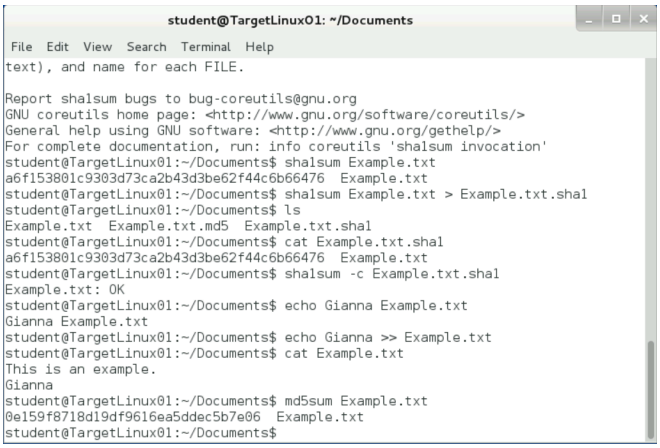
Screenshot 4: SHA1 hash value for `Example.txt` file.



```
student@TargetLinux01: ~/Documents
File  Edit  View  Search  Terminal  Help
      --status         don't output anything, status code shows success
  -w, --warn           warn about improperly formatted checksum lines

      --strict         with --check, exit non-zero for any invalid input
      --help     display this help and exit
      --version  output version information and exit

The sums are computed as described in FIPS-180-1.  When checking, the input
should be a former output of this program.  The default mode is to print
a line with checksum, a character indicating type (`*' for binary, ` ' for
text), and name for each FILE.

Report sha1sum bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'sha1sum invocation'
student@TargetLinux01:~/Documents$ sha1sum Example.txt
a6f153801c9303d73ca2b43d3be62f44c6b66476  Example.txt
student@TargetLinux01:~/Documents$ sha1sum Example.txt > Example.txt.sha1
student@TargetLinux01:~/Documents$ ls
Example.txt  Example.txt.md5  Example.txt.sha1
student@TargetLinux01:~/Documents$ cat Example.txt.sha1
a6f153801c9303d73ca2b43d3be62f44c6b66476  Example.txt
student@TargetLinux01:~/Documents$
```

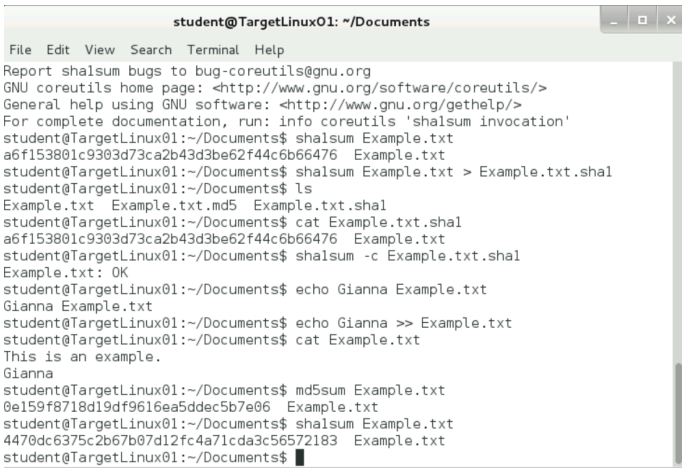
Screenshot 5: Verification that `Example.txt.sha1` was created, saved to Documents, and holds the sha1 hash for `Example.txt`.

Part 3: Modify a File and Verify Hash Values



```
student@TargetLinux01: ~/Documents
File  Edit  View  Search  Terminal  Help
text), and name for each FILE.

Report sha1sum bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'sha1sum invocation'
student@TargetLinux01:~/Documents$ sha1sum Example.txt
a6f153801c9303d73ca2b43d3be62f44c6b66476  Example.txt
student@TargetLinux01:~/Documents$ sha1sum Example.txt > Example.txt.sha1
student@TargetLinux01:~/Documents$ ls
Example.txt  Example.txt.md5  Example.txt.sha1
student@TargetLinux01:~/Documents$ cat Example.txt.sha1
a6f153801c9303d73ca2b43d3be62f44c6b66476  Example.txt
student@TargetLinux01:~/Documents$ sha1sum -c Example.txt.sha1
Example.txt: OK
student@TargetLinux01:~/Documents$ echo Gianna Example.txt
Gianna Example.txt
student@TargetLinux01:~/Documents$ echo Gianna >> Example.txt
student@TargetLinux01:~/Documents$ cat Example.txt
This is an example.
Gianna
student@TargetLinux01:~/Documents$ md5sum Example.txt
0e159f8718d19df9616ea5ddec5b7e06  Example.txt
student@TargetLinux01:~/Documents$
```

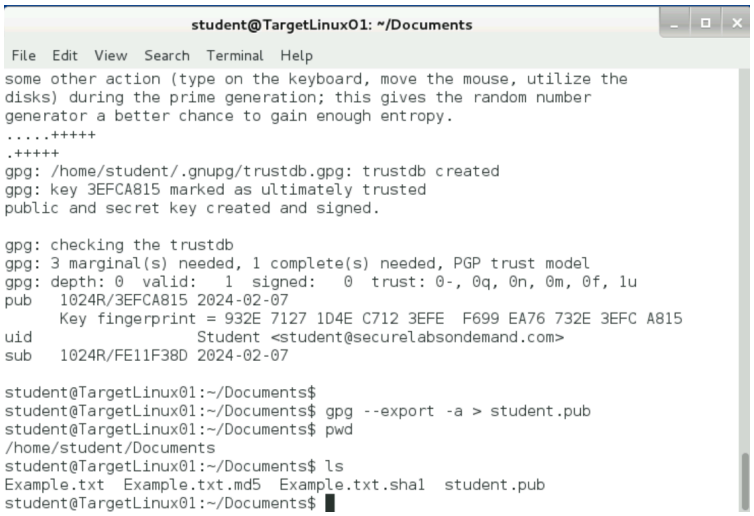
Screenshot 6: Verification of new MD5 hash after modifying the file.



```
student@TargetLinux01: ~/Documents
File  Edit  View  Search  Terminal  Help
Report sha1sum bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'sha1sum invocation'
student@TargetLinux01:~/Documents$ sha1sum Example.txt
a6f153801c9303d73ca2b43d3be62f44c6b66476  Example.txt
student@TargetLinux01:~/Documents$ sha1sum Example.txt > Example.txt.sha1
student@TargetLinux01:~/Documents$ ls
Example.txt  Example.txt.md5  Example.txt.sha1
student@TargetLinux01:~/Documents$ cat Example.txt.sha1
a6f153801c9303d73ca2b43d3be62f44c6b66476  Example.txt
student@TargetLinux01:~/Documents$ sha1sum -c Example.txt.sha1
Example.txt: OK
student@TargetLinux01:~/Documents$ echo Gianna Example.txt
Gianna Example.txt
student@TargetLinux01:~/Documents$ echo Gianna >> Example.txt
student@TargetLinux01:~/Documents$ cat Example.txt
This is an example.
Gianna
student@TargetLinux01:~/Documents$ md5sum Example.txt
0e159f8718d19df9616ea5ddec5b7e06  Example.txt
student@TargetLinux01:~/Documents$ sha1sum Example.txt
4470dc6375c2b67b07d12fc4a71cda3c56572183  Example.txt
student@TargetLinux01:~/Documents$
```

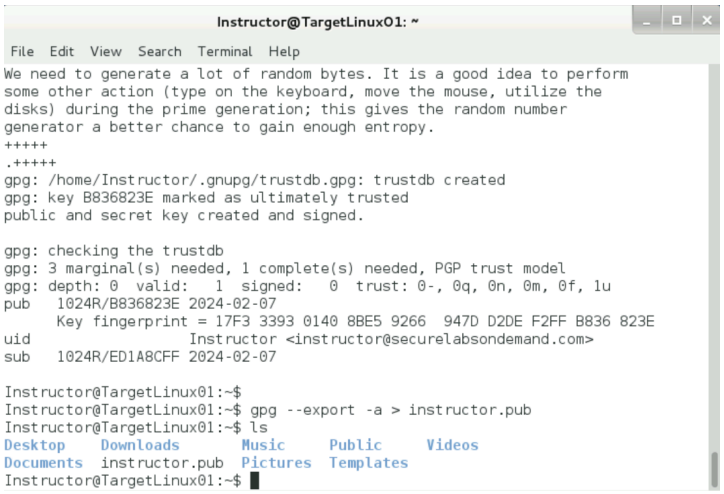
Screenshot 7: Verification of new SHA1 hash after modifying the file.

Part 4: Generate GnuPG Key



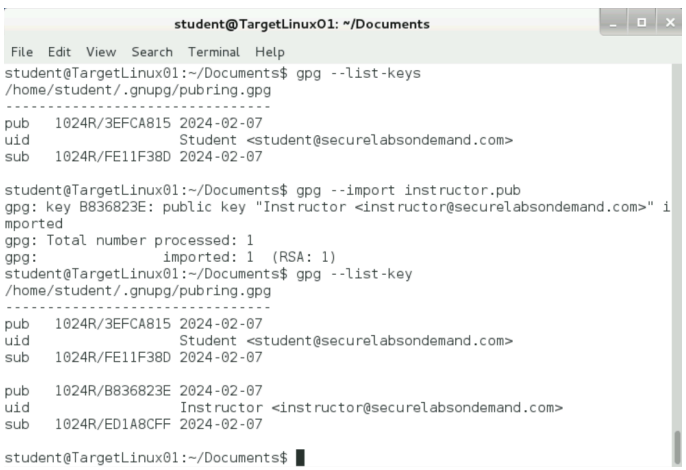
```
student@TargetLinux01: ~/Documents
File  Edit  View  Search  Terminal  Help
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
.....+++++
.+++++
gpg: /home/student/.gnupg/trustdb.gpg: trustdb created
gpg: key 3EFCA815 marked as ultimately trusted
public and secret key created and signed.

gpg: checking the trustdb
gpg: 3 marginal(s) needed, 1 complete(s) needed, PGP trust model
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
pub   1024R/3EFCA815 2024-02-07
      Key fingerprint = 932E 7127 1D4E C712 3EFE  F699 EA76 732E 3EFC A815
uid                  Student <student@securelabsondemand.com>
sub   1024R/FE11F38D 2024-02-07

student@TargetLinux01:~/Documents$
student@TargetLinux01:~/Documents$ gpg --export -a > student.pub
student@TargetLinux01:~/Documents$ pwd
/home/student/Documents
student@TargetLinux01:~/Documents$ ls
Example.txt  Example.txt.md5  Example.txt.sha1  student.pub
student@TargetLinux01:~/Documents$
```

Screenshot 8: Verification of new file `student.pub` in the Documents folder.



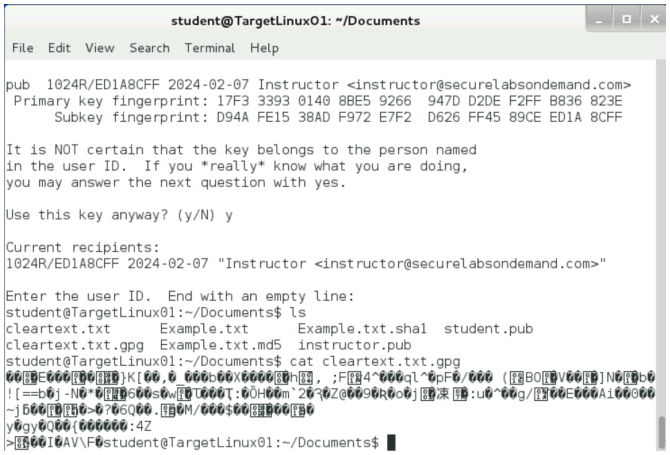
Screenshot 9: Verification of new file `instructor.pub` in the Instructor home folder.

Part 5: Share a GnuPG Key

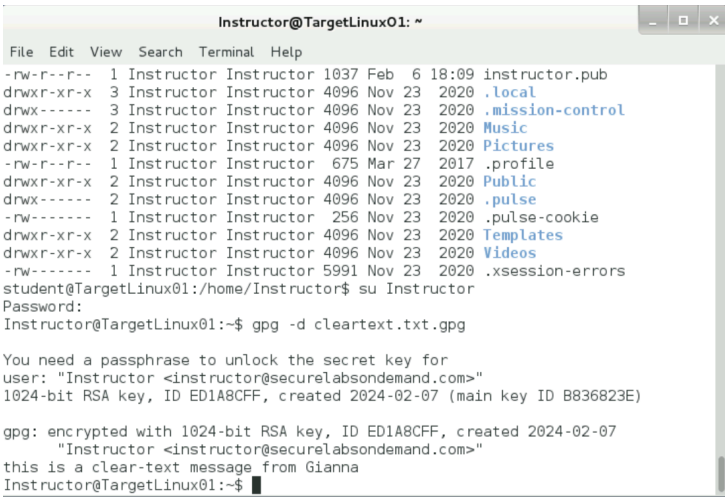


Screenshot 10: Verification of shared Instructor key.

Part 6: Encrypt and Decrypt a ClearText Message



Screenshot 11: Verification of encrypted file `cleartext.txt`.



Screenshot 12: Contents of decrypted file `cleartext.txt`.

SECTION 2 - APPLIED LEARNING

Part 1: Create a Text File on Linux

```
student@TargetLinux01:~$ cd Documents
student@TargetLinux01:~/Documents$ vi Example2.txt
student@TargetLinux01:~/Documents$ cat Example2.txt
This file is from Gianna.
student@TargetLinux01:~/Documents$
```

Screenshot 13: Verification of `Example2.txt` file creation.

Part 2: Create a MD5sum and a SHA1sum Hash String

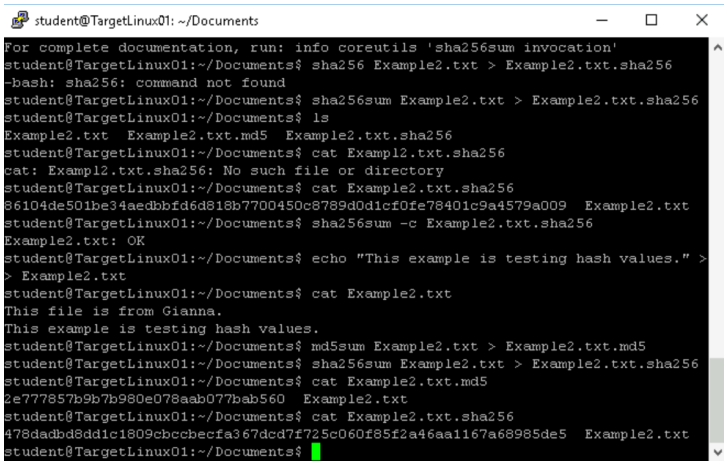
```
student@TargetLinux01: ~/Documents
      --status         don't output anything, status code shows success
  -w, --warn           warn about improperly formatted checksum lines

      --strict         with --check, exit non-zero for any invalid input
      --help     display this help and exit
      --version  output version information and exit

The sums are computed as described in RFC 1321.  When checking, the input
should be a former output of this program.  The default mode is to print
a line with checksum, a character indicating type (`*' for binary, ` ' for
text), and name for each FILE.

Report md5sum bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'md5sum invocation'
student@TargetLinux01:~/Documents$ md5sum Example2.txt
bb68e40986a6f2e1065d716cfe59ba74  Example2.txt
student@TargetLinux01:~/Documents$ md5sum Example2.txt > Example2.txt.md5
student@TargetLinux01:~/Documents$ ls
Example2.txt  Example2.txt.md5
student@TargetLinux01:~/Documents$ cat Example2.txt.md5
bb68e40986a6f2e1065d716cfe59ba74  Example2.txt
student@TargetLinux01:~/Documents$
```

Screenshot 14: Verification of MD5 hash in file `Example2.txt.md5`.

```
student@TargetLinux01: ~/Documents
      --strict         with --check, exit non-zero for any invalid input
      --help     display this help and exit
      --version  output version information and exit

The sums are computed as described in FIPS-180-2.  When checking, the input
should be a former output of this program.  The default mode is to print
a line with checksum, a character indicating type (`*' for binary, ` ' for
text), and name for each FILE.

Report sha256sum bugs to bug-coreutils@gnu.org
GNU coreutils home page: <http://www.gnu.org/software/coreutils/>
General help using GNU software: <http://www.gnu.org/gethelp/>
For complete documentation, run: info coreutils 'sha256sum invocation'
student@TargetLinux01:~/Documents$ sha256 Example2.txt > Example2.txt.sha256
-bash: sha256: command not found
student@TargetLinux01:~/Documents$ sha256sum Example2.txt > Example2.txt.sha256
student@TargetLinux01:~/Documents$ ls
Example2.txt  Example2.txt.md5  Example2.txt.sha256
student@TargetLinux01:~/Documents$ cat Examp12.txt.sha256
cat: Examp12.txt.sha256: No such file or directory
student@TargetLinux01:~/Documents$ cat Example2.txt.sha256
86104de501be34aedbbfd6d818b7700450c8789d0d1cf0fe78401c9a4579a009  Example2.txt
student@TargetLinux01:~/Documents$
```

Screenshot 15: Verification of creation and contents of `Example.txt.sha256`.

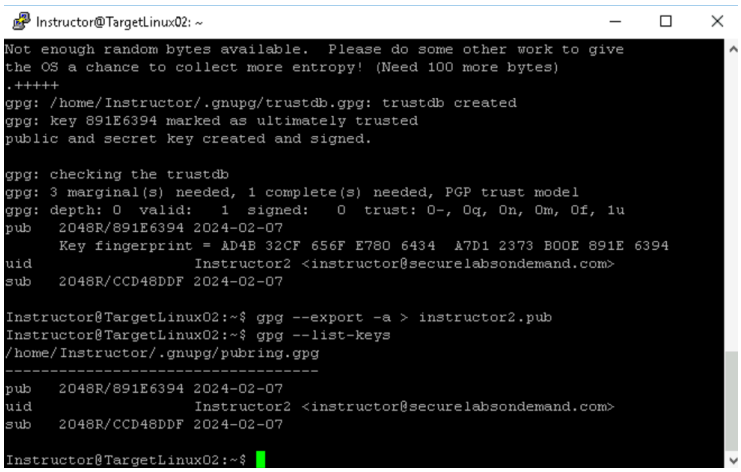
Part 3: Modify a File and Verify Hash Values



```
student@TargetLinux01: ~/Documents
For complete documentation, run: info coreutils 'sha256sum invocation'
student@TargetLinux01:~/Documents$ sha256 Example2.txt > Example2.txt.sha256
-bash: sha256: command not found
student@TargetLinux01:~/Documents$ sha256sum Example2.txt > Example2.txt.sha256
student@TargetLinux01:~/Documents$ ls
Example2.txt  Example2.txt.md5  Example2.txt.sha256
student@TargetLinux01:~/Documents$ cat Examp12.txt.sha256
cat: Examp12.txt.sha256: No such file or directory
student@TargetLinux01:~/Documents$ cat Example2.txt.sha256
86104de501be34aedbbfd6d818b7700450c8789d0d1cf0fe78401c9a4579a009  Example2.txt
student@TargetLinux01:~/Documents$ sha256sum -c Example2.txt.sha256
Example2.txt: OK
student@TargetLinux01:~/Documents$ echo "This example is testing hash values." >
> Example2.txt
student@TargetLinux01:~/Documents$ cat Example2.txt
This file is from Gianna.
This example is testing hash values.
student@TargetLinux01:~/Documents$ md5sum Example2.txt > Example2.txt.md5
student@TargetLinux01:~/Documents$ sha256sum Example2.txt > Example2.txt.sha256
student@TargetLinux01:~/Documents$ cat Example2.txt.md5
2e777857b9b7b980e078aab077bab560  Example2.txt
student@TargetLinux01:~/Documents$ cat Example2.txt.sha256
478dadbd8dd1c1809cbccbecfa367dcd7f725c060f85f2a46aa1167a68985de5  Example2.txt
student@TargetLinux01:~/Documents$
```

Screenshot 16: Verification of MD5 and SHA256 hash change after file modification.

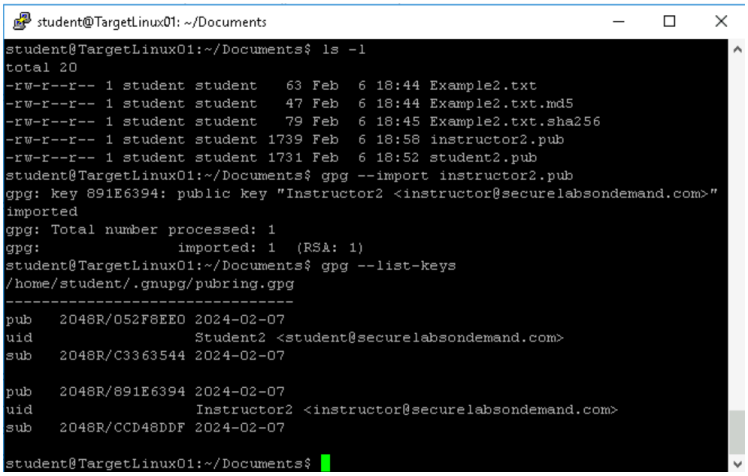
Part 4: Generate GnuPG Key



```
Instructor@TargetLinux02: ~
Not enough random bytes available.  Please do some other work to give
the OS a chance to collect more entropy! (Need 100 more bytes)
.+++++
gpg: /home/Instructor/.gnupg/trustdb.gpg: trustdb created
gpg: key 891E6394 marked as ultimately trusted
public and secret key created and signed.

gpg: checking the trustdb
gpg: 3 marginal(s) needed, 1 complete(s) needed, PGP trust model
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
pub   2048R/891E6394 2024-02-07
      Key fingerprint = AD4B 32CF 656F E780 6434  A7D1 2373 B00E 891E 6394
uid                  Instructor2 <instructor@securelabsondemand.com>
sub   2048R/CCD48DDF 2024-02-07

Instructor@TargetLinux02:~$ gpg --export -a > instructor2.pub
Instructor@TargetLinux02:~$ gpg --list-keys
/home/Instructor/.gnupg/pubring.gpg
---------------------------------
pub   2048R/891E6394 2024-02-07
uid                  Instructor2 <instructor@securelabsondemand.com>
sub   2048R/CCD48DDF 2024-02-07

Instructor@TargetLinux02:~$
```

Screenshot 17: Verification of Instructor's public key.

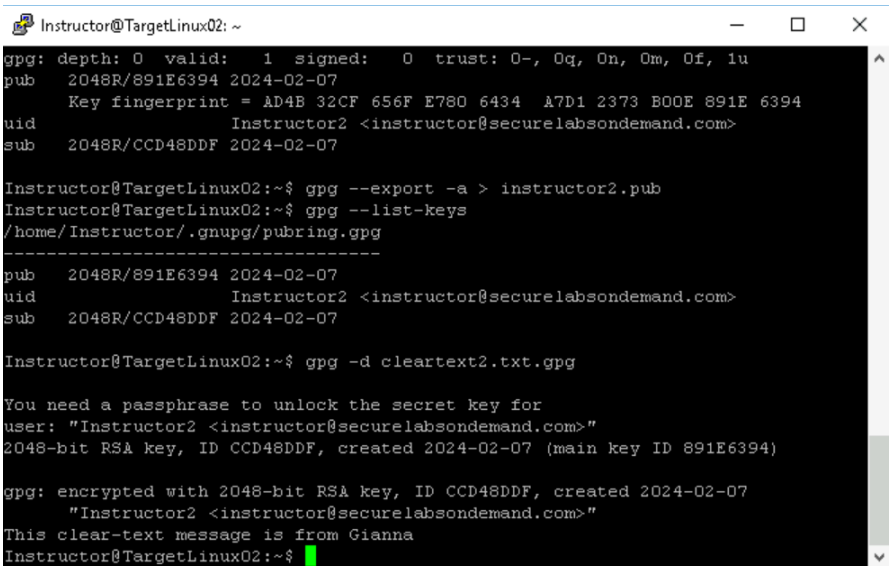
Part 5: Share a GnuPG Key



```
student@TargetLinux01: ~/Documents
student@TargetLinux01:~/Documents$ ls -l
total 20
-rw-r--r-- 1 student student   63 Feb  6 18:44 Example2.txt
-rw-r--r-- 1 student student   47 Feb  6 18:44 Example2.txt.md5
-rw-r--r-- 1 student student   79 Feb  6 18:45 Example2.txt.sha256
-rw-r--r-- 1 student student 1739 Feb  6 18:58 instructor2.pub
-rw-r--r-- 1 student student 1731 Feb  6 18:52 student2.pub
student@TargetLinux01:~/Documents$ gpg --import instructor2.pub
gpg: key 891E6394: public key "Instructor2 <instructor@securelabsondemand.com>"
imported
gpg: Total number processed: 1
gpg:               imported: 1  (RSA: 1)
student@TargetLinux01:~/Documents$ gpg --list-keys
/home/student/.gnupg/pubring.gpg
--------------------------------
pub   2048R/052F8EE0 2024-02-07
uid                  Student2 <student@securelabsondemand.com>
sub   2048R/C3363544 2024-02-07

pub   2048R/891E6394 2024-02-07
uid                  Instructor2 <instructor@securelabsondemand.com>
sub   2048R/CCD48DDF 2024-02-07

student@TargetLinux01:~/Documents$
```

Screenshot 18: Verification of shared public keys on TargetLinux01 student account.

Part 6: Encrypt and Decrypt a ClearText Message



```
Instructor@TargetLinux02: ~
gpg: depth: 0  valid:   1  signed:   0  trust: 0-, 0q, 0n, 0m, 0f, 1u
pub   2048R/891E6394 2024-02-07
      Key fingerprint = AD4B 32CF 656F E780 6434  A7D1 2373 B00E 891E 6394
uid                  Instructor2 <instructor@securelabsondemand.com>
sub   2048R/CCD48DDF 2024-02-07

Instructor@TargetLinux02:~$ gpg --export -a > instructor2.pub
Instructor@TargetLinux02:~$ gpg --list-keys
/home/Instructor/.gnupg/pubring.gpg
---------------------------------
pub   2048R/891E6394 2024-02-07
uid                  Instructor2 <instructor@securelabsondemand.com>
sub   2048R/CCD48DDF 2024-02-07

Instructor@TargetLinux02:~$ gpg -d cleartext2.txt.gpg

You need a passphrase to unlock the secret key for
user: "Instructor2 <instructor@securelabsondemand.com>"
2048-bit RSA key, ID CCD48DDF, created 2024-02-07 (main key ID 891E6394)

gpg: encrypted with 2048-bit RSA key, ID CCD48DDF, created 2024-02-07
      "Instructor2 <instructor@securelabsondemand.com>"
This clear-text message is from Gianna
Instructor@TargetLinux02:~$
```

Screenshot 19: Verification of decrypted cleartext file.